

AstraNotes — Week 7.2 Lab

Testing Strategy and First Test Set

Jonathan Fong | CSEN 296B | May 18, 2026

Part 1 — AstraNotes Testing Strategy

Three test levels are used throughout AstraNotes development. Unit tests isolate a single class — `NoteFactory`, `NoteManager`, `SecureNote` — and substitute injected mocks for any collaborator. Integration tests exercise two or more real classes together: `FileStorage` round-trips through `NoteFactory` and `NoteManager` against a per-test temporary directory, and `SecureNote` serialization runs through `FileStorage` with a mock `EncryptionEngine`. Feature-level tests would drive `CLIView` end to end and validate user-facing workflows, but none appear in this first set; the CLI shell is thin enough that everything reachable through it is also reachable at a lower level with cleaner failure modes.

The injection boundary is what makes this practical. `StorageInterface` and `EncryptionEngine` are pure abstractions so unit tests can substitute mocks — SPR-2 requires it, and the Definition of Done's testability gate enforces it on every header file. No unit test in this project links a real crypto library or writes to the real filesystem.

Timing: unit tests run pre-commit on every change. Integration tests run pre-merge against the feature branch. The full suite — unit and integration combined — runs pre-sprint-review before any backlog item is marked done.

Part 2 — First Test Set

Feature A — US-01 Note Creation

Targets the implementations from Week 6's Note Creation slice. Bodies below follow GTest's `TEST()` macro convention; helper construction is elided for brevity.

Test ID	Level	What It Verifies	Maps To
A1	Unit	A valid title and type produce a <code>TextNote</code> with a generated UUID, correct type, and stamped timestamps	US-01 AC-1, FR-1
A2	Unit	<code>NoteFactory</code> throws <code>std::invalid_argument</code> on empty title; collection unchanged	US-01 AC-2, FR-1a
A3	Unit	<code>NoteFactory</code> throws on whitespace-only title	FR-1a
A4	Unit	<code>NoteFactory</code> throws on a title exceeding 255 characters	FR-1a
A5	Unit	Two notes created with identical titles receive distinct UUIDs	US-01 AC-3
A6	Integration	<code>NoteManager.add</code> transfers ownership; <code>findByUUID</code> retrieves the same note by key	NFR-1

Test ID	Level	What It Verifies	Maps To
A7	Integration	findByUUID returns nullptr for an unknown UUID without exception or crash	NFR-2

Test bodies

A1 — CreatesTextNoteWithValidTitle

```
TEST(NoteFactoryTest, CreatesTextNoteWithValidTitle) {
    NoteFactory factory;
    auto note = factory.create(NoteType::Text, "Sprint Planning");
    ASSERT_NE(note, nullptr);
    EXPECT_EQ(note->getType(), NoteType::Text);
    EXPECT_FALSE(note->getUUID().empty());
    EXPECT_EQ(note->getTitle(), "Sprint Planning");
}
```

A2 — RejectsEmptyTitle

```
TEST(NoteFactoryTest, RejectsEmptyTitle) {
    NoteFactory factory;
    EXPECT_THROW(factory.create(NoteType::Text, ""),
                 std::invalid_argument);
}
```

A3 — RejectsWhitespaceOnlyTitle

```
TEST(NoteFactoryTest, RejectsWhitespaceOnlyTitle) {
    NoteFactory factory;
    EXPECT_THROW(factory.create(NoteType::Text, "  "),
                 std::invalid_argument);
    EXPECT_THROW(factory.create(NoteType::Text, "\t\n"),
                 std::invalid_argument);
}
```

A4 — RejectsTitleExceeding255Chars

```
TEST(NoteFactoryTest, RejectsTitleExceeding255Chars) {
    NoteFactory factory;
    std::string longTitle(256, 'x');
    EXPECT_THROW(factory.create(NoteType::Text, longTitle),
                 std::invalid_argument);
}
```

A5 — GeneratesDistinctUUIDsForDuplicateTitles

```
TEST(NoteFactoryTest, GeneratesDistinctUUIDsForDuplicateTitles) {
    NoteFactory factory;
    auto n1 = factory.create(NoteType::Text, "Same Title");
    auto n2 = factory.create(NoteType::Text, "Same Title");
    EXPECT_NE(n1->getUUID(), n2->getUUID());
}
```

A6 — AddInsertsAndFindByUUIDRetrieves

```
TEST(NoteManagerTest, AddInsertsAndFindByUUIDRetrieves) {
    NoteManager manager;
    NoteFactory factory;
    auto note = factory.create(NoteType::Text, "Hello");
    UUID id = note->getUUID();
    manager.add(std::move(note));
    const Note* found = manager.findByUUID(id);
}
```

```

    ASSERT_NE(found, nullptr);
    EXPECT_EQ(found->getUUID(), id);
}

```

A7 — FindByUUIDReturnsNullForMissingUUID

```

TEST(NoteManagerTest, FindByUUIDReturnsNullForMissingUUID) {
    NoteManager manager;
    EXPECT_EQ(manager.findByUUID("nonexistent-uuid"), nullptr);
}

```

Feature B — US-05 SecureNote Passphrase Gate (outlines)

Tests below inject a mock EncryptionEngine into SecureNote at construction time. The mock returns canned ciphertext on encrypt() and either canned plaintext or a configurable failure on decrypt() — no real crypto library is linked, satisfying SPR-2.

Test ID	Level	What It Verifies	Maps To
B1	Unit	Correct passphrase invokes mock.decrypt and the returned plaintext is delivered to the caller	US-05 AC-1, FR-5
B2	Unit	Wrong passphrase returns the wrong-passphrase error code; no plaintext is exposed in the result	US-05 AC-2, FR-5a
B3	Unit	Empty passphrase is rejected before decryption; the mock asserts decrypt was never invoked	US-05 AC-3, FR-5a
B4	Unit	A passphrase shorter than 8 characters returns the invalid-input error code, distinct from wrong-passphrase	FR-5
B5	Unit	After access completes, no plaintext is reachable through any public accessor on SecureNote	US-05 AC-1, SPR-1
B6	Integration	FileStorage serializes a SecureNote and the resulting JSON record contains the ciphertext field only — never the plaintext	US-05 AC-4, SPR-1

Feature C — US-04 JSON Persistence (outlines)

Tests run against a per-test temporary directory created via a GTest fixture and removed on teardown. Real filesystem access is permitted at the integration level; SPR-2's no-filesystem rule applies to unit tests only.

Test ID	Level	What It Verifies	Maps To
C1	Integration	A note saved then reloaded round-trips with UUID, title, type, and timestamps intact	US-04 AC-1, FR-4
C2	Integration	A JSON file containing one TextNote, one VoiceNote, and one SecureNote loads into three concrete subtypes via NoteFactory	US-04 AC-2, FR-4

Test ID	Level	What It Verifies	Maps To
C3	Integration	Pointing FileStorage at a non-existent file returns an empty collection; the next save creates the file	FR-4a
C4	Integration	A corrupt notes.json is renamed to notes.json.quarantine.<timestamp>; load returns empty; no crash	US-04 AC-3, FR-4b
C5	Integration	A JSON file with three valid records and one malformed record loads the three valid ones and skips the bad one	FR-4b
C6	Integration	A JSON file containing two records that share a UUID loads the first record and discards the second	FR-4

Feature D — US-06 Title Substring Search (outlines)

Tests inject a strict mock StorageInterface that fails the test on any save/load/remove invocation, proving FR-6's no-I/O constraint during search. The note collection is populated directly in the test fixture.

Test ID	Level	What It Verifies	Maps To
D1	Unit	Searching for a substring returns every note whose title contains it, regardless of position	US-06 AC-1, FR-6
D2	Unit	A query with no matches returns an empty collection — no exception, no error code	US-06 AC-2, FR-6
D3	Unit	Search is case-insensitive: query "ALPHA" matches a note titled "alpha"	FR-6
D4	Unit	An empty query string returns every note in the collection	FR-6
D5	Unit	searchByTitle never invokes save/load/remove on the injected mock StorageInterface	US-06 AC-3

Part 3 — Test Level and Timing

A1 through A5 sit at unit level because NoteFactory has no collaborator — it allocates a Note and returns it. A6 and A7 are integration because NoteManager and NoteFactory cooperate. The SecureNote tests B1 through B5 remain unit-level: the EncryptionEngine boundary is mocked so the test exercises one class at a time. B6 is integration because FileStorage participates in the serialization path. The C-series is integration throughout — FileStorage, NoteFactory, and a real (temporary-directory) filesystem all participate in every round-trip. D1 through D5 are unit because search runs entirely on the in-memory collection, and the strict mock storage proves the absence of I/O rather than introducing collaboration.

Unit tests (A1–A5, B1–B5, D1–D5) run pre-commit on every change. Integration tests (A6, A7, B6, C1–C6) run pre-merge against the feature branch. The full suite runs pre-sprint-review before any backlog item is marked done.

Part 4 — AI Use Reflection

AI helped enumerate candidate tests against each acceptance criterion and the refined baseline, surfacing edge cases — whitespace-only titles, partially-corrupt files, the empty-query rule for search — that map cleanly to FR-1a, FR-4b, and FR-6. The role was generation and gap-critique against the existing AC list, not authority over what belongs in the first set.

One thing surfaced during the review worth recording: Week 6 implements NoteManager's note collection as `std::map`, while NFR-1 specifies `unordered_map` for $O(1)$ average lookup. Test A6 was deliberately written to check behavior (correct retrieval by key) rather than complexity class, so it remains valid against either implementation; the data-structure mismatch is logged here for a future sprint to reconcile.

What I kept, changed, or rejected from AI output: AI initially scoped the test set to US-01 and US-05 only; I rejected that narrowing and directed the expansion to also cover US-04 (persistence) and US-06 (search), so the first set spans the behaviors Sprint 1 and Sprint 3 will need to verify rather than only what's already implemented.